Chapter 1 Summary

Davide Colabove

UAV Software Engineering: Chapter 1 Overview

This document serves as a brief summary of the key concepts covered in Chapter 1, designed for quick reference on the portfolio website.

1 Memory Management and RAII

In critical UAV systems, memory leaks are unacceptable. Modern C++ employs the **RAII (Resource Acquisition Is Initialization)** paradigm. Memory allocation occurs in the constructor, and deallocation happens in the destructor. When an object goes out of scope, memory is automatically freed, ensuring consistent execution times for high-frequency control loops.[cite: 3]

Smart Pointers

We explored `std::unique_ptr`, which grants exclusive ownership of a heap-allocated memory block without adding overhead compared to raw pointers. Because ownership is exclusive, it cannot be copied, only moved via `std::move()`, preventing dangling references and double-free errors.[cite: 3] Modern C++ best practices dictate using `std::make_unique` for exception safety. Other system components can read data safely without taking ownership using the `get()` method.[cite: 3]

2 Concurrency and Thread Safety

Multicore environments introduce the risk of **data races**, where multiple threads access the same memory location simultaneously (with at least one write operation). In a flight controller, this can lead to corrupted data (like altitude readings) and erratic motor commands.[cite: 1]

Synchronization Primitives

To ensure thread safety, C++ provides several tools:

- `std::mutex`**:** Ensures mutual exclusion for critical sections. To avoid deadlocks (where a lock is acquired but never released), we use `std::lock_guard<std::mutex>` to apply RAII to mutex locking, guaranteeing exception-safe unlocking.[cite: 1]
- `std::atomic`**:** For lock-free, hardware-level atomic operations on simple variables.[cite: 1]
- `std::condition_variable`**:** Used to block a thread until a specific event occurs, often paired with a mutex.[cite: 1]

Producer-Consumer Architecture

To handle asynchronous data generation (e.g., from sensors) and processing, we implemented the **Producer-Consumer pattern** using a thread-safe `std::queue` (FIFO container) and a `std::condition_variable`. This ensures telemetry packets are buffered and processed sequentially without data loss or wasted CPU cycles.[cite: 1]

3 Build Systems: CMake

Managing large C++ projects (like ROS 2 workspaces) requires robust build systems. **CMake** is the industry standard—a cross-platform generator that reads high-level rules (`CMakeLists.txt`) and produces low-level build files (like Makefiles).[cite: 2]

Out-of-Source Builds

To keep the source code directory clean, we enforce **out-of-source builds**:

1. Create an isolated directory: `mkdir build && cd build`.[cite: 2]
2. Generate build files targeting the parent directory: `cmake ...`[cite: 2]
3. Compile the binary: `make`.[cite: 2]

The `build/` directory should always be added to `.gitignore` to prevent compiled binaries from polluting the version control history.[cite: 2]